



Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3067 Redes

Laboratorio 3

Protocolos de Enrutamiento

1 Antecedentes

Conociendo a dónde enviar los mensajes para cualquier router se vuelve trivial el envío de mensajes. Únicamente es necesario conocer el destino final y se reenvía al vecino que puede proveer la mejor ruta al destino. Toda esa información es almacenada en las tablas de enrutamiento.

No obstante, con el dinamismo con el que se espera que pueda funcionar el Internet es necesario que dichas tablas puedan actualizarse y acomodarse a cambios en la infraestructura. Los algoritmos con los que se actualizan estas tablas son conocidos como protocolos de enrutamiento.

2 Objetivos

- Conocer los protocolos de enrutamiento utilizados en las implementaciones actuales de Internet.
- Comprender cómo funcionan las tablas de enrutamiento.
- Implementar los protocolos de enrutamiento y probarlos en una red simulada sobre un protocolo de capa superior.
- Analizar el funcionamiento de los protocolos de enrutamiento.

3 Desarrollo

Los protocolos de enrutamiento funcionan sobre nodos interconectados entre sí, donde cada nodo conoce únicamente cuáles son los vecinos que tiene. Dicha información inicial será proporcionada para cada nodo.

A partir de ello, se levantarán varias instancias (nodos) de las implementaciones de los algoritmos que se describen más adelante, para proceder a enviar mensajes y simular una red.

Nuestro medio de desarrollo será nuestra computadora local y probaremos los algoritmos utilizando sockets (similar al laboratorio 2). En la fase de pruebas utilizaremos una red privada (Tailscale) como el medio para nuestra red, y utilizando sockets, por lo que cada IP corresponderá a un usuario/nodo.

Para tales efectos se conformarán grupos de 2 integrantes (deben ser los mismos del laboratorio 2) y luego se conformarán topologías de tres parejas (seis integrantes en total).

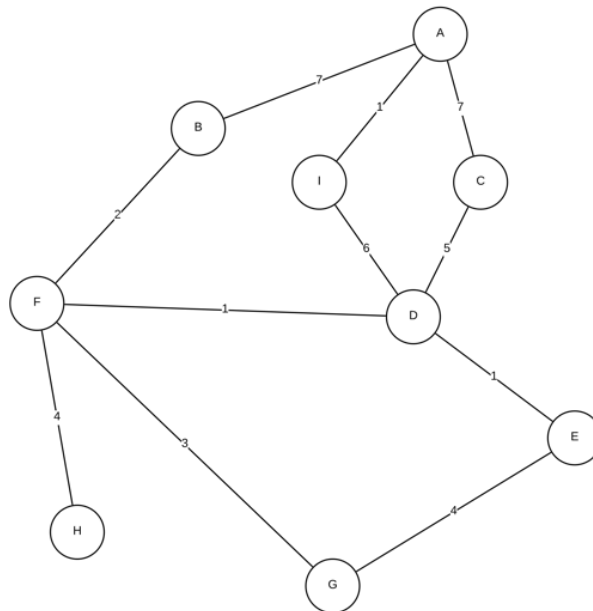


Imagen 1: Mapa de conexiones entre nodos.

En esta propuesta, cada uno de los nodos corresponde a un cliente el cuál puede enviar o recibir mensajes para conformar su tabla de enrutamiento y posteriormente enviar un paquete de un nodo origen a un nodo destino utilizando dicha tabla. Cada nodo corresponde a un proceso independiente que se comunica vía Sockets hacia el siguiente nodo.

El laboratorio consiste de dos partes principales: (1) la implementación de los protocolos y (2) la interconexión y prueba de esos algoritmos. Las implementaciones deben de estar debidamente identificadas y publicadas en un repositorio, junto con sus requerimientos para su uso e instalación. Las implementaciones idealmente deben de poder ejecutarse en distintas plataformas de forma sencilla (makefiles, python venv, npm run, .jar).

3.1 Plano de control

En cada nodo (router) se deberá implementar el protocolo Link State de manera completa (consulta de vecinos, paquetes HELLO, recibimiento de costos de enlace, construcción del LSA, uso del algoritmo flooding para enviar el LSA a los vecinos, construcción del grafo de la red y cálculo de las rutas más cortas. Se debe construir un archivo `nodo_tabla_enrutamiento.csv` donde esté la información de la tabla de ruteo que será utilizada por el plano de datos.

3.2 Plano de datos y arquitectura de capas

Una vez construidas las tablas de enrutamiento para cada nodo, se podrán enviar mensajes de un nodo a otro que deben tener un formato similar al siguiente (serializado): {nodo_origen, nodo_destino, mensaje}.

Los nodos deben tener una capa de red, donde, al recibir un mensaje, extraigan únicamente el nodo destino y, utilizando el archivo `nodo_tabla_enrutamiento.csv`, obtengan la IP y el puerto correspondientes y abran una conexión vía sockets por donde se reenviará el mensaje.

Además, se deberá añadir la capa detección y corrección de errores del laboratorio 2 (utilice Hamming 7,4) a los nodos, de modo que su funcionamiento será:

1. Recibimiento de una cadena de bits 1000101111001
2. Detección y corrección de errores (aplica a todos los bits)
3. Extracción de los bits de datos
4. Deserialización de los bits correspondientes al destino
5. Consulta de tabla de ruteo
6. Inicio de conexión mediante sockets según la IP y puerto devueltos por la tabla de ruteo
7. Serialización de los bits del destino
8. Aplicación de Hamming 7,4 a todos los bits
9. Envío de los bits a través del socket

Finalmente, se debe indicar qué nodo será el cliente y que nodo será el servidor. Estos nodos no serán routers, pero se debe establecer comunicación mediante sockets entre el cliente y su nodo router, que funcionará como puerta de enlace predeterminada, y entre el servidor y su nodo router, que funcionará como puerta de enlace predeterminada.

3.3 Conexión y Pruebas de los algoritmos

La segunda parte del laboratorio consiste en conectar y probar nuestros algoritmos implementados. Usaremos una red privada virtual (Tailscale). Utilicen sus cuentas personas de Google. Esta plataforma permite un uso gratuito de hasta seis usuarios por red. Una persona debe crear su red e invitar a otras a unirse. Se trabajará cada topología con 3 parejas.

La idea es que cada red de tres parejas implemente su protocolo de Link State, y el formato estándar de los mensajes de datos entre el nodo cliente y el nodo servidor. Si las tres parejas siguen el protocolo, sus implementaciones deberían funcionar sin problema. . La base es la siguiente, pudiendo agregar elementos si así lo consideran (**ojo, deben ponerse de acuerdo entre todas las parejas si modifican el protocolo. Esto significa que debería haber interoperabilidad entre codigos de los distintos grupos**). La estructura será tipo JSON. Por ejemplo, para un paquete LSA:

```
{
  "type": "LSA",
  "origin": self.self_ip,
  "seq": self.seq,
  "links": links,
  "from": self.self_ip,
}
```

La implementación de la capa de detección y corrección de errores solo aplica a los nodos cuando ya hayan finalizado el cálculo de las tablas de ruteo.

3.3 Otros detalles para las pruebas y conexiones

El día de la entrega probaremos las topologías en clase. Las pruebas corresponderán al laboratorio 4.

Al iniciar un nodo, este solicitará la configuración y procederá a descubrir a sus vecinos. El nodo tendrá dos procesos/hilos en simultáneo: el forwarding y el routing. Todo debe correr en paralelo/asíncrono mediante el uso de hilos, procesos, tiempos de espera, etc .

El objetivo es lograr que los algoritmos se estabilicen y los mensajes pasen por los nodos que corresponden a la ruta óptima.

4 Rúbrica de evaluación

Elemento		Ponderación	
Código		75%	
	Documentación, orden, comentarios, limpieza, legibilidad/funcionalidad balanceada, etc..		5%
	Implementación de los Algoritmos, de forma eficiente y optimizada.		70%
Reporte Escrito		25%	
	Encabezado, Ortografía, Formato Adecuado, Descripción de la Práctica		2.5%
	Descripción de los Algoritmos Utilizados y su Implementación		10%
	Resultados		5%
	Discusión		5%
	Conclusiones + Comentarios + Referencias		2.5%

** Una inasistencia injustificada anula la nota del laboratorio.

Entregar en Canvas:

1. Archivo .pdf con su reporte en grupo
2. Codigo utilizado para el Laboratorio, en un rar si es necesario
3. Link a su repositorio